AuditTailor

Quantified-Preference, Source-Grounded and Self-Verifying Personalized Slide Generation from Scientific Papers

Raksha Muthyala USN: 24BTRCL226
Nannam Sai Divya Naina USN: 24BTRCL136
Department of Computer Science and Engineering
Faculty of Engineering, Jain University
Bengaluru, India

Abstract

Automatic paper-to-slides systems have recently begun to take the presenter into account: given one earlier paper–slides pair and a slide template, SlideTailor infers the presenter's habits and applies them to a new paper. In that line of work the inferred preference is a free-text profile, so adherence can only be judged indirectly; the finished deck is not checked against the profile or repaired; and generated bullets are not checked cheaply against the source paper or for text fit inside slide boxes. We propose **AuditTailor**, a lightweight framework with three parts: (i) a *Quantified Preference Profile* (QPP) that turns a reference deck into numbers (section emphasis, section order, text density, and bullet, figure and equation usage), together with a budgeted planner that converts those numbers into a slide-per-section plan; (ii) a *verify-and-repair loop* driven by an auditable preference-distance score; and (iii) *grounding and fit checks*, in which every bullet carries a pointer to a source passage and the deck is scored for source support, numeric fidelity and overflow. The framework has two drafting modes: an *extractive* mode (TF-IDF scoring with redundancy reduction) that needs no language model and no API key, and an *LLM* mode for fluent abstractive bullets. We also propose a small open dataset, **PrefSlides-Mini**, and an evaluation protocol that pairs deterministic metrics with a cross-family LLM judge and a small human study. The pipeline is easy to read and needs only standard Python libraries; we tested its main components end to end and confirmed that they run in the key-free mode.

1 Introduction

Two people asked to present the same paper will produce very different decks. One spends half of the slides on experiments, keeps text to a few words, and fills the rest with plots. Another walks slowly through the background in full-sentence bullets and compresses the results into a single table. Neither deck is wrong; they reflect different habits of selection, ordering, density and visual style. Most paper-to-slides systems ignore this and treat the task as one fixed conversion from document to deck [5, 6, 2]. Persona-Aware-D2S allows customization but only along a small, fixed set of preference categories [7].

SlideTailor [1] reformulates the problem: the presenter supplies one earlier paper–slides pair (which implicitly shows *how* they present) and a `.pptx` template (which shows *how the deck should look*), and the system generates a deck for a new paper that follows both. This is a natural way to specify preferences, and the paper reports gains over strong baselines under both LLM-based and human evaluation. Reading it closely, however, suggests three directions in which the approach can be extended.

A measurable preference profile. As described in the paper, the inferred profile is a structured textual description produced by LLM calls. Adherence is evaluated afterwards: SlideTailor reports a coverage score (overlap of LLM-extracted topics), a flow score (order similarity of those topics, using a normalized Levenshtein distance [13]) and 1–5 scores from an LLM judge [1]. These are useful, but the profile itself holds no quantities (such as the share of slides given to each section) that could be used

both to plan a deck and to audit it afterwards. We borrow the flow idea for our order distance and add distances over slide allocation and style statistics; coverage and flow are not new to us.

Feedback on the generated deck. As we read the paper, the pipeline is a one-way sequence of stages, and we found no step that detects or corrects a deck that drifts from the profile. The authors themselves note that their LLM judge is more generous than human raters and shows self-preference, which makes an automatic check based on the same kind of judge less trustworthy still.

Lightweight grounding and fit checks. Slide generators can state things the paper does not support, and can produce slides whose text overflows its box. SlideTailor's own qualitative analysis points at related failures in other systems, such as generated images that do not come from the paper and slides with large unused areas or leftover template placeholders [1]. Linking slide content back to its source has been studied: Maheshwari et al. generate presentations with attribution of each slide's content to the source document, using a graph neural network combined with an LLM [3]. We do *not* present attribution as a new idea. Our narrower aim is a simple, deterministic version (a retrieval-based pointer per bullet plus numeric and fit checks) that is cheap enough to run inside a generation loop and to use as a repair signal for a presenter-conditioned system.

Objectives of this study. We ask whether an explicit numeric profile with budgeted planning adheres more closely to the presenter than a free-text profile; whether a verify-and-repair loop improves adherence at a small extra cost; whether grounding and fit checks reduce unsupported bullets, unsupported numbers and text overflow; whether the automatic metrics agree with human judgments; and how much quality is lost when drafting without any language model.

Contributions. (1) The Quantified Preference Profile and a budgeted section planner (Sections 4.2–4.3). (2) A preference-distance score and a verify-and-repair loop (Section 4.5). (3) Source pointers with automatic support, numeric-fidelity and overflow checks inside that loop (Sections 4.4–4.5). (4) A key-free extractive drafting mode, which also serves as a transparent baseline (Section 4.4). (5) The PrefSlides-Mini dataset plan and an evaluation protocol that avoids relying on a single self-judging LLM (Section 4.6).

2 Proposed Work

We propose the framework below. Its core is a small set of numbers (the profile), a planner that uses them, and a checker that measures the finished deck with the same numbers. The main text keeps only the few equations that carry the idea; full metric definitions, the notation table (Table 3) and the dataset plan are in the appendices.

The idea in brief. The framework rests on four simple ideas. (1) A presenter's habits can be summarised as a probability distribution over sections (how the slides are shared), an ordering of sections, and a few style rates. (2) Planning is interpolation: the slide budget for the new paper lies between what the presenter would do and what the paper itself suggests, and a single weight decides where. (3) Checking is measurement: the generated deck is described with the same numbers, and the distance between the two descriptions says how faithfully the presenter was followed. (4) Repair is a bounded search that keeps the best deck seen so far, so it can only improve or leave the result unchanged.

3 Related Work

3.1 Paper-to-Slides Generation and Personalization

Early systems treated slide generation as summarization. D2S frames it as query-based text summarization of a paper into slide text [5], and DOC2PPT generates slides from scientific documents with both text and figures in mind [6]. PPTAgent takes an edit-based approach: it studies a reference presentation to derive slide categories and content structures, plans an outline, and then generates editing actions applied to chosen reference slides, which keeps the original layouts and styles; it is evaluated on content, design and coherence [2]. AutoPresent generates slides from scratch as code [9]. Related generation tasks extend beyond slides to posters [8] and to presentation videos, including a benchmark that pairs 101 papers with the videos and slides their authors made [4].

Personalization enters with Persona-Aware-D2S, which adapts slides to a small set of fixed audience types [7], and with SlideTailor, which learns open-ended preferences from a paper–slides pair and a template [1]. SlideTailor distils a content profile and an aesthetic profile, plans slides with a reorganizer, an outline designer and a template selector, and edits the template to produce an editable `.pptx`. It also contributes a benchmark (PSP) with 200 target papers, 50 sample pairs and 10 templates, and metrics for coverage, flow, content structure and aesthetics. Its stated limitations include restriction to scientific papers, the absence of end-to-end training, and a noticeable gap between LLM-judge scores and human ratings [1].

3.2 Attribution, Extractive Selection and Evaluation

Maheshwari et al. argue that slides draw on scattered parts of a document and that LLMs struggle with long inputs, and propose a graph-based method that produces presentations with attribution of content to the source [3]. Our grounding check is intentionally simpler and is not a competitor to that method; it is a verification signal.

For the key-free drafting mode we use classical extractive selection: TF-IDF representations [11] and Maximal Marginal Relevance (MMR), which re-ranks candidates by trading relevance against similarity to items already chosen [10]. We treat this mode as a transparent baseline and a fallback, not as a state-of-the-art contribution.

LLM-as-a-judge is now common but is known to have biases [12], and SlideTailor itself reports self-bias. We therefore pair a judge from a *different model family* than the generator with deterministic measurements (Section 4.6). For order similarity we reuse the normalized Levenshtein distance [13].

3.3 Positioning Relative to Prior Work

Among the papers we examined for this proposal (our search was limited, not systematic), we did not find one that combines (i) an explicit numeric presenter profile used both for planning and for auditing, (ii) a repair loop driven by that audit, and (iii) deterministic per-bullet grounding and fit checks in the same loop. This reflects our reading of the literature rather than a proven novelty claim, and it should be rechecked with a broader search. Table 1 summarizes the comparison; entries reflect what is stated in the cited papers, not an audit of their code.

	Preference input	Numeric audit	Feedback loop	Source attribution
D2S / DOC2PPT [5, 6]	none	–	–	–
PPTAgent [2]	template deck	–	iterative edits	–
Persona-Aware-D2S [7]	4 fixed types	–	–	–
SlideTailor [1]	pair + template	LLM scores	–	–
Attribution method [3]	none	–	–	GNN + LLM
AuditTailor (proposed)	pair + template	**numeric**	**repair loop**	**retrieval pointers**

Table 1: Positioning of the proposal relative to prior work. "–" means not described in the cited paper.

4 Proposed Work

We propose the framework below. Its core is a small set of numbers (the profile), a planner that uses them, and a checker that measures the finished deck with the same numbers. The main text keeps only the few equations that carry the idea; full metric definitions, the notation table (Table 3) and the dataset plan are in the appendices.

The idea in brief. The framework rests on four simple ideas. (1) A presenter's habits can be summarised as a probability distribution over sections (how the slides are shared), an ordering of sections, and a few style rates. (2) Planning is interpolation: the slide budget for the new paper lies between what the presenter would do and what the paper itself suggests, and a single weight decides where. (3) Checking is measurement: the generated deck is described with the same numbers, and the distance between the two descriptions says how faithfully the presenter was followed. (4) Repair is a bounded search that keeps the best deck seen so far, so it can only improve or leave the result unchanged.

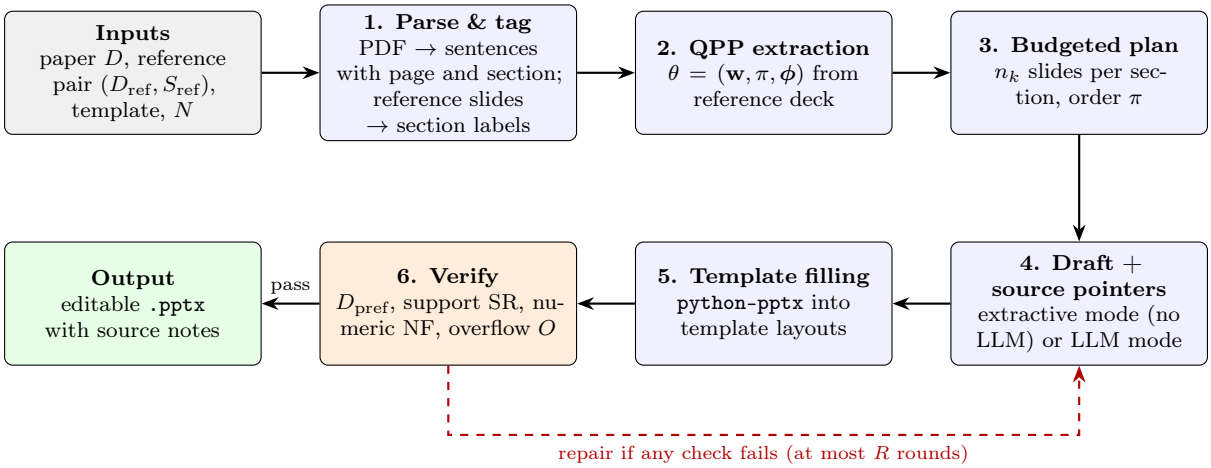


Figure 1: AuditTailor pipeline. Solid arrows are the forward pass; the dashed arrow is the verify-and-repair loop. Stages 3 and 5 and the checks of Stage 6 are deterministic code; Stage 1 is rule-based with an optional LLM fallback for tagging, and repair rewrites may call an LLM in LLM mode.

4.1 Problem Formulation and Overview

Let D be a scientific paper (PDF), $(D_{\text{ref}}, S_{\text{ref}})$ a paper–slides pair that shows the presenter's habits, S_{tmpl} a `.pptx` template, and N the desired number of content slides. We extend the formulation of SlideTailor [1] with an explicit slide budget:

$$S = F\big(D,\ (D_{\text{ref}}, S_{\text{ref}}),\ S_{\text{tmpl}};\ N\big). \tag{1}$$

The output S should (a) present D faithfully with traceable sources, (b) follow the presenter's habits as encoded in the pair, (c) follow the template, and (d) render without overflow. Figure 1 shows the six stages. Stages 1–2 read the inputs and build the QPP. Stage 3 plans the deck. Stage 4 drafts bullets either in extractive mode (no language model) or in LLM mode, and attaches a source pointer to each bullet. Stage 5 fills the template. Stage 6 scores the deck and, if needed, triggers a repair pass back to Stage 4. Everything except LLM-mode drafting, the optional fact sheet, the optional LLM fallback for section tagging and some repair rewrites is ordinary code.

4.2 Stages 1–2: Quantified Preference Profile

Goal: Turn the presenter's reference deck into a small set of numbers that can be inspected, used to plan a new deck, and used again to audit it.

Approach: *Parsing.* The paper is read with a PDF library. Section headings are found from lines set in bold or larger type (the library exposes font size and style flags) and from numbered-heading patterns; running headers, footers, captions and the reference list are removed; text is joined across hyphenation and split into sentences by a rule-based splitter. Each sentence is stored with its page number and section label, so it can later be cited as (page, section, sentence id).

Section tagging. We use $K = 6$ canonical section labels: Motivation, Background, Method, Setup, Results, Conclusion; title, outline and reference slides are excluded. Each section of the target paper D and each slide of S_{ref} gets one label. Headings and slide titles are matched against a small keyword table (for example "introduction" and "motivation" map to Motivation; "related work" and "background" to Background; "method" and "approach" to Method; "experimental setup" and "datasets" to Setup; "results" and "analysis" to Results; "conclusion" and "future work" to Conclusion). A slide whose title matches nothing inherits the label of the previous slide; in LLM mode an LLM may label such slides instead. Keyword collisions need care: in our prototype, "Experimental Results" was first mislabelled as Setup because of the word "experimental", which the priority order of the keyword table must prevent. We will check tagging accuracy against hand labels (Section 4.6).

Profile. From the tagged reference deck with N_{ref} slides, let c_k be the number of slides labelled k. The profile is

$$\theta = (\mathbf{w},\ \pi,\ \boldsymbol{\phi}), \qquad w_k = \frac{c_k}{\sum_{j=1}^{K} c_j}, \tag{2}$$

where $\mathbf{w} \in \Delta^{K-1}$ is the *emphasis* distribution over sections, π is the sequence of section labels in order of first appearance (the *flow*), and $\boldsymbol{\phi} = (\eta, \beta, \rho, \varepsilon)$ collects four style statistics averaged over content slides: the *fill ratio* η (estimated rendered text height divided by text-box height, averaged over text boxes, so it does not depend on the template's absolute box sizes); β, the fraction of content slides whose body text has three or more separate paragraphs (our simplest operational test for a bulleted style, using the text-frame paragraphs exposed by `python-pptx`); ρ, the fraction of slides containing at least one picture or table; and ε, the fraction of slides containing an equation (detected by a simple pattern for math symbols and equation objects; a heuristic that will be validated on the reference decks). A fifth quantity, the mean number of bullets per content slide $\bar{b}$, is also read from the reference deck; it only sets bullet budgets in Stage 4 and is not part of the style distance.

Intuition: the profile answers three questions about a presenter: where the slides are spent ($\mathbf{w}$), in what order (π), and in what visual style ($\boldsymbol{\phi}$).

Success criteria: The feature extractor runs on every reference deck and its four style features match hand counts on a spot check; the section tagger is accurate against two annotators' labels (Section 4.6).

4.3 Stage 3: Budgeted Planning

Goal: Convert the presenter's emphasis into an integer number of slides per section for the *new* paper, without ignoring what the paper itself contains.

Approach: Let ℓ_k be the length in words of section k of the target paper and $P = \{k : \ell_k > 0\}$ the sections that actually occur in D. A reference deck may emphasise a section the paper does not have, so the emphasis is restricted to P and renormalised, $w'_k = w_k / \sum_{j \in P} w_j$ (if that sum is zero, $w'_k = p_k$). With $p_k = \ell_k / \sum_{j \in P} \ell_j$ the natural share, the planner blends the presenter's emphasis with the paper's own structure:

$$q_k = \lambda\, w'_k + (1 - \lambda)\, p_k, \qquad k \in P, \quad \lambda \in [0, 1]. \tag{3}$$

It then converts Nq_k into integer slide counts n_k by the largest-remainder rule: each section receives $\lfloor Nq_k \rfloor$ slides, and the $m = N - \sum_{j \in P} \lfloor Nq_j \rfloor$ sections with the largest fractional remainders receive one extra slide each (ties go to the larger q_k). A section with $n_k = 0$ is simply not presented (it then lowers coverage). The slide order follows π; sections of π that are not in P are skipped, and a section in P that is absent from π is inserted before the Conclusion. The parameter λ controls how strongly the presenter's habit overrides the paper's structure ($\lambda = 0$ ignores the presenter; $\lambda = 1$ copies the reference allocation) and is tuned on a development split.

Theoretical properties. Two facts make the planner predictable. First, the blend $\mathbf{q}$ is the least-squares compromise between the presenter's emphasis and the paper's natural shares: it is the point on the line between them that minimises the weighted squared distance to both, so λ has a clear meaning as a trade-off weight. Second, the largest-remainder rounding always returns exactly N slides in total, and no section's count differs from its ideal share Nq_k by a full slide or more, because the fractional parts add up to the number of extra slides handed out.

Worked toy example. Take four merged sections (Setup and Results merged into Experiments, all present in the paper, so $w' = w$), a reference deck of $N_{\text{ref}} = 12$ slides with counts $(2, 3, 5, 2)$, a target paper whose sections have $(1200, 2400, 1800, 600)$ words, $\lambda = 0.6$ and $N = 10$. Table 2 gives the result. The planned allocation $(0.2, 0.3, 0.4, 0.1)$ lies between the presenter's emphasis and the paper's natural structure, and its distance to the reference emphasis is $D_{\text{alloc}} = \frac{1}{2}\|\mathbf{n}/N - \mathbf{w}\|_1 \approx 0.083$ (Eq. (5)).

Section	c_k	w_k	p_k	Nq_k	n_k
Background	2	0.167	0.20	1.8	2
Method	3	0.250	0.40	3.1	3
Experiments	5	0.417	0.30	3.7	4
Conclusion	2	0.167	0.10	1.4	1
Total	12	1.000	1.00	10.0	10

Table 2: Toy example of the planner with $\lambda = 0.6$, $N = 10$. Floors are $(1, 3, 3, 1)$, so $m = 2$ and the two largest remainders (Background 0.8, Experiments 0.7) receive the extra slides. This is an illustration, not an experimental result.

Success criteria: The planned allocation is closer to the presenter's emphasis than the paper's natural structure is, for $\lambda > 0$, and the slide counts always sum to N.

4.4 Stage 4: Drafting, Source Pointers and Fit

Goal: Produce the bullets of each planned slide, each with a source pointer, in a mode that needs no language model and in a mode that reads fluently.

Approach: *Key-free extractive mode.* For section k the system must produce $n_k\bar{b}$ bullets, where $\bar{b}$ is the mean number of bullets per slide in the reference deck (rounded; 4 if the reference deck has none). If a section has fewer usable sentences than $n_k\bar{b}$, all of them are used and the unmet quota passes to the section with the next-highest q_k. Every candidate sentence s_i of D (body sentences only; the title, abstract, captions and references are excluded) gets a relevance score $\mathrm{Rel}(s_i)$ between 0 and 1 that combines four simple signals: similarity to the title and abstract, mean keyword weight, the section weight from the presenter-conditioned plan, and a rule-based evidence indicator (Appendix B). Within each section, sentences are then chosen one at a time by MMR [10]:

$$s^* = \arg\max_{s_i \in C_k \setminus A} \Big[\lambda_m \mathrm{Rel}(s_i) - (1-\lambda_m) \max_{s_j \in A} \cos(\mathbf{v}_i, \mathbf{v}_j)\Big], \tag{4}$$

where C_k is the candidate set of section k, A the sentences already selected, $\mathbf{v}$ the TF-IDF vectors, and $\lambda_m \in [0,1]$ the relevance–diversity trade-off. When $A = \emptyset$ the maximum is defined as 0, so the first pick is simply the highest-scoring sentence. *Intuition:* pick the most useful sentence, then penalise anything that repeats what is already on the slide. Selected sentences are lightly trimmed by rules (drop citations and parentheticals, cut at a clause boundary to fit the box) and become bullets. A toy MMR run is given in Appendix C. The mode uses no language model and no API key, and it also serves as a transparent baseline.

LLM mode. For each section k, one LLM call receives the section text, the plan (n_k slides), the style targets from $\boldsymbol{\phi}$ (for instance, "mostly bullets, short phrases, include the main figure"), an optional *fact sheet* (up to M key results with their numbers extracted from D) and a per-box character limit, and returns slide text. The model can be a free-tier hosted model or an open-weights model run locally; the framework does not depend on a particular one.

Source pointers. Every bullet u receives a pointer $\mathrm{ptr}(u) = (\text{page}, \text{section}, \text{sentence id})$. In extractive mode the pointer is the sentence it was taken from. In LLM mode it is the paper sentence with the highest TF-IDF cosine similarity to the bullet. Pointers are written into the slide's notes field, and the references slide lists the source paper.

Template filling. Each planned slide is matched to a template layout by role (title, text, text+image) with a simple rule, and text, figures and tables extracted from the paper are inserted with `python-pptx`. Placeholders left unfilled are removed, which addresses the leftover-placeholder problem noted for existing systems [1]. The first version supports a few simple layouts rather than arbitrary templates.

Fit check. For each text box b, the rendered height of its text is estimated by wrapping the string with the chosen font's metrics; the per-box fill is $\eta_b = h_{\text{text}}(b)/h_{\text{box}}(b)$ and the deck-level overflow rate $O(S)$ is the share of boxes with $\eta_b > 1$ (taken as 0 if there are no text boxes). Overflowing boxes are shortened (by rules in extractive mode, by an LLM call in LLM mode); if that is not enough, the font size is reduced by at most two steps; anything still overflowing is flagged. Because this is an estimate, final overflow figures in the evaluation are also confirmed by rendering a sample of decks to PDF and checking visually.

Success criteria: Every bullet has a pointer; no unfilled placeholder remains; and the estimated overflow rate is zero or the remaining boxes are flagged.

4.5 Stage 6: Verify-and-Repair Loop

Goal: Measure the finished deck with the same numbers that built the plan, and repair it when it drifts from the presenter's profile or fails a grounding or fit check.

Approach: The generated deck is measured with the same code that was used on the reference deck, giving its own allocation, order and style rates. Three distances, each between 0 and 1, compare them with the profile: an *allocation distance* (half the total difference between the two share vectors, i.e. the share of slides that would have to move to another section), an *order distance* (the normalized edit distance between the two section orders [13]) and a *style distance* (the average gap in the four style rates, each scaled by how much that rate varies across reference decks, floored at 0.1, and capped at 1). They are combined into the preference distance

$$D_{\text{pref}} = \alpha_a D_{\text{alloc}} + \alpha_o D_{\text{order}} + \alpha_s D_{\text{style}}, \qquad \alpha_a + \alpha_o + \alpha_s = 1, \tag{5}$$

with default weights 1/3 each and a sensitivity analysis. The generated deck's allocation and order are always obtained by running the same section tagger on the generated slides (never read off the plan), so that baselines and AuditTailor are measured identically. The preference-faithfulness score reported later is $\mathrm{PF} = 1 - D_{\mathrm{pref}}$.

Theoretical properties. Every distance lies between 0 and 1, so D_{pref} and PF are bounded and easy to compare across decks; $D_{\mathrm{pref}} = 0$ exactly when the allocation, the order and all style rates match the reference. The allocation distance is the total-variation distance between two distributions, and the order distance cannot exceed 1 because an edit distance is never larger than the combined length of the two sequences.

Three further checks use the pointers of Stage 4 (definitions in Appendix B). *Source support* SR is the share of bullets whose best-matching paper sentence is similar enough to the bullet; it is a cheap lexical heuristic that does not prove entailment, so it is validated against manual judgments, and in extractive mode it is high by construction. *Numeric fidelity* NF is the share of numbers on the slides that also occur in the paper; it is conservative, since a number computed from two paper values is flagged even if correct, so flagged items go to a short manual review. *Overflow* O is the share of text boxes whose estimated text does not fit. The objective used to compare candidate decks across rounds (the stopping test uses thresholds, not J) adds them to the preference distance, with all weights defaulting to 1:

$$J(S) = D_{\mathrm{pref}} + \mu\, O + \nu\,(1 - \mathrm{NF}) + \xi\,(1 - \mathrm{SR}). \tag{6}$$

Repair step. If $D_{\mathrm{pref}} > \tau$, $O > 0$, $\mathrm{NF} < \kappa$ or $\mathrm{SR} < \sigma$, the loop applies one repair action per round, in a fixed priority order: overflow first, then low support or fidelity, then preference drift. For preference drift it picks the component with the largest weighted distance and applies a targeted fix: for allocation, the section with the largest slide-count gap gets slides added or merged by re-running Stage 4 for that section; for order, slides are re-sorted by code to follow π (no model needed); for style, only slides whose features deviate most are rewritten (for example, shortened, or converted between prose and bullets). For low support or fidelity, the offending bullets are replaced by the next-best candidate sentence (extractive mode) or redrafted with an instruction to use only fact-sheet numbers and the pointed passage (LLM mode). Algorithm 1 gives the full procedure. The loop runs at most R repair rounds (default $R = 2$), so at most $R + 1$ decks are scored, and it returns the scored deck with the lowest J. Since the first draft is among the scored decks, the returned deck's J is never higher than that of the first draft (a bounded local search that cannot make the result worse), and the loop terminates after at most $R + 1$ scoring passes.

Algorithm 1: AuditTailor with verify-and-repair

Input: D, $(D_{\mathrm{ref}}, S_{\mathrm{ref}})$, template, N, mode $\in \{\text{extractive}, \text{LLM}\}$, λ, τ, κ, σ, R
Output: deck $S^{\star}$

$\theta \leftarrow \mathrm{QPP}(D_{\mathrm{ref}}, S_{\mathrm{ref}})$;
$(\mathbf{n}, \pi) \leftarrow \textsc{Plan}(D, \theta, N, \lambda)$; `// Eq. (3)`
$S^{(0)} \leftarrow \textsc{DraftAndFill}(D, \mathbf{n}, \pi, \theta, \mathrm{mode})$; `// Eq. (4) or LLM; pointers`
for $r = 0, 1, \ldots, R$ **do**
  compute D_{pref}, O, NF, SR, J for $S^{(r)}$;
  if *($D_{pref} \le \tau$ **and** $O = 0$ **and** $\mathrm{NF} \ge \kappa$ **and** $\mathrm{SR} \ge \sigma$) **or** $r = R$* **then break**;
  $S^{(r+1)} \leftarrow \textsc{Repair}(S^{(r)})$;
return $S^{\star} = \arg\min_r J(S^{(r)})$

Success criteria: Mean PF improves after repair at a small extra cost, with at most two extra drafting calls per section on average, and the grounding and fit measures do not get worse.

Illustrative repair example

The following invented example shows one pass of the loop in extractive mode; it is an illustration, not a result. Suppose the paper reports "accuracy rises from 0.76 to 0.87" and a draft bullet reads "Accuracy improves by about 15%". The number 15 does not occur in the paper, so NF falls below κ and the bullet is flagged. The repair step swaps it for the next-best candidate sentence of the same section, "Accuracy rises from 0.76 to 0.87", whose numbers all occur in the source. On the next scoring pass NF returns to 1, the pointer now names the original sentence, and the deck is accepted. A reviewer who sees the first

bullet flagged can also accept it manually, since $(0.87 - 0.76)/0.76 \approx 14.5\%$ is a correct derived value; this is why flagged numbers go to short manual review rather than being counted as errors automatically.

4.6 Dataset and Experimental Design

Dataset (PrefSlides-Mini). SlideTailor's authors state that they release code and data: the code repository carries an MIT licence and the dataset is hosted separately on Hugging Face. We have not yet inspected the dataset's contents or terms of use. If they allow it, we reuse its papers, reference pairs and templates; otherwise, or to fill gaps, we build a small set ourselves. The benchmark spans several fields, whereas we restrict ourselves to NLP/ML papers to keep the study manageable. **PrefSlides-Mini** is intentionally small so that two or three students can finish it: 30 open-access target papers, 6 reference pairs, 4 templates, 60 evaluation triples (10 for development, 50 for testing), and manual annotation of section labels, support judgments and human ratings (Appendix D, Table 5). We begin with a pilot of about 5 papers to debug the pipeline before the full set. There is no gold deck for a given target paper, so evaluation is reference-*conditioned* for preference adherence (the reference is the presenter's deck, not a gold output) and reference-free for quality. Quality checks cover PDF parsing (manual inspection of all 30 papers), the section tagger (accuracy and annotator agreement) and the feature extractor (spot checks on every reference deck).

Systems compared.

- **B0 Plain extractive:** TF-IDF and MMR, uniform section weights, no profile (the key-free baseline).
- **B1 Generic LLM:** unconditioned slide generation (no reference pair), template only.
- **B2 SlideTailor:** the authors' released code (MIT-licensed repository) if it runs within our budget, after checking its API requirements; otherwise a prompt-level re-implementation, clearly labelled as such.
- **B3 PPTAgent:** template-based generation with the released PPTAgent code [2].
- **B4 Free-text profile + our drafting:** our LLM drafting and fill stages but with a free-text profile instead of the QPP, hence without the budgeted plan and the preference-driven repair (isolates the effect of the numeric profile).
- **AuditTailor-E** (extractive, with QPP, planner and loop) and **AuditTailor-L** (LLM mode), plus ablations: without budgeted planning; without repair loop; without source and fidelity checks; without fit check.

B1–B4 and AuditTailor-L use the same generator model, chosen on a development split from a free-tier hosted model or a locally run open-weights model. If no language model is available at all, B0 and AuditTailor-E still allow the preference-adherence and repair-loop questions, and part of the grounding question, to be tested.

Metrics. Deterministic metrics carry the main claims; LLM-judge scores are secondary (Appendix E, Table 6): PF and its parts, Coverage and Flow following [1], redundancy, SR, NF, O, judge scores, human ratings, and cost. The judge uses a different model family from the generator to reduce the self-bias reported for same-family judging [1, 12].

Hypotheses and success criteria. We state them before running experiments.

- **Preference adherence.** AuditTailor has higher mean PF than the free-text-profile baseline B4 (LLM mode) and than the plain extractive baseline B0 (extractive mode), with a paired significance test.
- **Repair loop.** The repair loop improves mean PF over the same system without it while adding at most two extra drafting calls per section on average.
- **Grounding and fit.** With source and fidelity checks, AuditTailor-L has higher mean automatic SR and NF and lower mean O than B1–B3, and a higher manual support rate than B4 on the manually checked sample.
- **Metric validity.** The automatic support score agrees with manual judgments (reported as accuracy and Cohen's κ [14]), and the cross-family judge correlates positively with human ratings (Spearman, with confidence interval). If either is weak, we report that as a finding.

- **Cost of key-free drafting.** AuditTailor-E has lower human readability ratings than AuditTailor-L but comparable preference adherence PF; we report the size of the gap.

A hypothesis that is not supported will be reported as such. This is still a useful outcome, because it tells us which component of the pipeline does not matter.

Protocol. Every system runs on the same 60 triples (30 papers $\times$ 2 random choices of reference pair and template), of which 10 form a development split and 50 the test split. LLM-based systems are run twice with different sampling seeds and averaged per triple; extractive systems are deterministic and run once. The manual studies (support check and human ratings) use 10 test triples and four systems (B0, B4, AuditTailor-E, AuditTailor-L). We report mean $\pm$ standard deviation, bootstrap 95% confidence intervals, and paired Wilcoxon signed-rank tests [16] over triples with Holm correction [17]. Hyperparameters (the blend weight λ, the MMR trade-off λ_m, the weights in the relevance score and in D_{pref}, the thresholds $\tau, \kappa, \sigma, t_s$, the round limit R, and the objective weights μ, ν, ξ) are tuned on the 10 development triples only.

Guarding against metric overfitting. PF is also what the repair loop reduces, so a high PF alone cannot show that decks are better. We therefore compute PF for every baseline with the same tagger, and also report Coverage and Flow, the cross-family judge scores and the human structure-match ratings, none of which are minimised by the loop.

Optional experiment: number of reference decks. If at least three presenters have three or more public decks, we also build the profile from $k \in \{1, 2, 3\}$ decks of the same presenter (averaging $\mathbf{w}$ and taking the medoid flow π) and measure how PF changes with k. Otherwise we only report sensitivity to which single reference pair is used.

4.7 Implementation Plan and Feasibility

The work is split into six phases that fit a student project: parsing (PyMuPDF), profile and planner (Python, `python-pptx`), extractive mode (scikit-learn, NumPy), verifier (Python, Pillow), LLM mode (free-tier or local open model), and evaluation (LibreOffice rendering, SciPy); the detailed plan and an equation-to-code map are in Appendix F. The key-free path is built first, so that a working end-to-end system exists before any language model is added. No GPU training is required.

As a feasibility check we implemented the profile extraction, planner, MMR selection, pointers, and the support, numeric-fidelity, overflow and distance computations in about 150 lines of Python (NumPy, scikit-learn, `python-pptx`, Pillow). The code reproduces the toy numbers of Table 2 and Appendix Table 4 exactly ($n_k = (2, 3, 4, 1)$, $D_{\text{alloc}} \approx 0.083$, MMR picks s_1, s_3, s_4) and runs end to end on a synthetic paper and reference deck. Randomised tests also confirm the planner and distance properties numerically. This is a feasibility check only; it is not an experimental result.

4.8 Evaluation and Limitations

The proposal has limits that we state openly. Attribution to source content is not new [3]; our claim is limited to a simple, deterministic version inside a presenter-conditioned loop. Our reading of the literature was limited, so the combination we propose has not been shown to be new. The preference score PF depends on the accuracy of our rule-based section tagger, and it is also the quantity that the repair loop reduces, so it is reported together with measures that do not share this dependency. Extractive mode copies source sentences, so slides may read like paper prose and its support score is high by construction. The automatic support score is lexical and can accept a bullet that is not truly entailed, which is why we validate it manually. A single reference pair may be a noisy sample of a presenter's habits; we test sensitivity by swapping reference pairs. The five-feature style description ignores colour and typography, which the template handles. The ε feature, the numeric-fidelity check and the fit estimate are heuristics, and the fit estimate does not replace rendering in real PowerPoint. Numeric fidelity checks numbers only; it cannot tell whether a correct number is attached to the right claim, so human ratings of content accuracy remain necessary. LLM drafting can introduce errors into technical content, so generated decks must be checked by a person before use. The study covers NLP/ML papers only; other fields are future work.

Only public, appropriately licensed papers and slides will be used, and no human-subject data beyond annotator ratings is collected; annotators will be informed and credited. All experiments will be run with at least two seeds for LLM systems and reported with confidence intervals. Code, section labels, profiles and evaluation scripts will be released once licences are confirmed.

5 Conclusion

We propose AuditTailor, a framework that makes presenter preferences measurable, checks generated decks against them, and adds deterministic grounding and fit checks to a short repair loop. A key-free extractive mode makes it implementable without any language-model access and doubles as a transparent baseline, while an LLM mode targets fluent slides. The contribution is not a bigger model but a more auditable pipeline, and the evaluation design allows each claim to be tested separately, including the possibility that some components do not help.

References

[1] W. Zeng, M. Ouyang, L. Cui, and H. T. Ng. SlideTailor: Personalized presentation slide generation for scientific papers. *arXiv preprint arXiv:2512.20292*, 2025.

[2] H. Zheng, X. Guan, H. Kong, W. Zhang, J. Zheng, W. Zhou, H. Lin, Y. Lu, X. Han, and L. Sun. PPTAgent: Generating and evaluating presentations beyond text-to-slides. In *Proc. EMNLP 2025*, pages 14413–14429, 2025.

[3] H. Maheshwari, S. Bandyopadhyay, A. Garimella, and A. Natarajan. Presentations are not always linear! GNN meets LLM for text document-to-presentation transformation with attribution. In *Findings of EMNLP 2024*, pages 15948–15962, 2024.

[4] Z. Zhu, K. Q. Lin, and M. Z. Shou. Paper2Video: Automatic video generation from scientific papers. *arXiv preprint arXiv:2510.05096*, 2025.

[5] E. Sun, Y. Hou, D. Wang, Y. Zhang, and N. X. Wang. D2S: Document-to-slide generation via query-based text summarization. In *Proc. NAACL-HLT 2021*, pages 1405–1418, 2021.

[6] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song. DOC2PPT: Automatic presentation slides generation from scientific documents. In *Proc. AAAI 2022*, pages 634–642, 2022.

[7] I. Mondal, S. Shwetha, A. Natarajan, A. Garimella, S. Bandyopadhyay, and J. Boyd-Graber. Presentations by the humans and for the humans: Harnessing LLMs for generating persona-aware slides from documents. In *Proc. EACL 2024*, pages 2664–2684, 2024.

[8] W. Pang, K. Q. Lin, X. Jian, X. He, and P. Torr. Paper2Poster: Towards multimodal poster automation from scientific papers. *arXiv preprint arXiv:2505.21497*, 2025.

[9] J. Ge, Z. Z. Wang, X. Zhou, Y.-H. Peng, S. Subramanian, Q. Tan, M. Sap, A. Suhr, D. Fried, G. Neubig, et al. AutoPresent: Designing structured visuals from scratch. In *Proc. CVPR 2025*, pages 2902–2911, 2025.

[10] J. Carbonell and J. Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proc. SIGIR 1998*, pages 335–336, 1998.

[11] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[13] L. Yujian and L. Bo. A normalized Levenshtein distance metric. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1091–1095, 2007.

[14] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[15] J. Cohen. Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[16] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[17] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

A Notation

Symbol	Meaning
D, $(D_{\text{ref}}, S_{\text{ref}})$, S_{tmpl}, N	target paper; reference paper–slides pair; template; number of content slides
K, k, P	number of section labels (6); a label; labels that occur in D
$\mathbf{w}$, $\mathbf{w}'$, π	presenter's slide-share emphasis; its restriction to P, renormalised; section order
$\boldsymbol{\phi} = (\eta, \beta, \rho, \varepsilon)$, $\bar{b}$	style statistics (fill ratio; fractions of bulleted, figure and equation slides); bullets per slide
p_k, q_k, n_k, λ	paper's natural share; blended share; slides given to section k; blend weight
Rel	extractive relevance score of a sentence (four weighted terms, Appendix B)
λ_m, A, C_k	MMR trade-off; sentences already selected; candidates of section k
$D_{\text{alloc}}, D_{\text{order}}, D_{\text{style}}, D_{\text{pref}}$, PF	preference distances in $[0, 1]$; $\text{PF} = 1 - D_{\text{pref}}$
SR, NF, O, t_s	source support, numeric fidelity, overflow rate; similarity threshold for support
J, τ, κ, σ, R	loop objective; stopping thresholds; maximum number of repair rounds

Table 3: Main notation.

B Metric Definitions

Relevance score. In extractive mode each candidate sentence receives a relevance score between 0 and 1, a weighted average of four terms whose weights are non-negative and sum to 1:

- *Topical similarity:* the TF-IDF cosine similarity between the sentence and the title plus abstract (IDF is computed over the sentences of the paper, each treated as a document);
- *Keyword weight:* the mean of the TF-IDF weights of the sentence's terms, scaled by the largest such value in the paper; averaging over the sentence length stops long sentences from winning automatically (the term is 0 if the maximum is 0);
- *Section weight:* the blended share q_k of the sentence's section divided by the largest q_j, taken from the *presenter-conditioned plan*. This term is constant inside a section, so it does not change the ranking within a section; the profile acts mainly through the slide budgets n_k, and this term matters only when sentences of different sections compete (the quota hand-over and the ablation without a plan). We will report whether its weight has any measurable effect;
- *Evidence indicator:* a rule-based 0/1 flag that is 1 when the sentence contains a number together with a result word (for example "accuracy", "improves", "%").

Fit check. With $\eta_b = h_{\text{text}}(b)/h_{\text{box}}(b)$ the fill of text box b and B the set of text boxes,

$$O(S) = \frac{1}{|B|} \sum_{b \in B} \mathbb{1}[\eta_b > 1], \tag{7}$$

taken as 0 if there are no text boxes.

Source support. Let U be the set of bullets in the deck. A bullet counts as supported when its best-matching paper sentence is similar enough:

$$\text{SR}(S) = \frac{1}{|U|} \sum_{u \in U} \mathbb{1}\Big[\max_{s_j \in D} \cos(\mathbf{v}_u, \mathbf{v}_j) \geq t_s\Big]. \tag{8}$$

This automatic score is a cheap heuristic: lexical similarity does not prove that a claim is entailed by the passage. We therefore validate it against manual judgments on a sample of bullets (Section 4.6) and report their agreement. In extractive mode SR is high by construction, so there it serves only as a reference point. The threshold t_s is chosen on the development bullets that have manual support labels, as the value that maximises agreement with those labels; $\text{SR} = 1$ if $U = \emptyset$.

Numeric fidelity. Let $\mathcal{N}(S)$ be the set of normalized numeric tokens found in the slide text (excluding slide numbers) and $\mathcal{N}(D)$ the set found in the paper, normalized for formatting such as decimals and percent signs:

$$\text{NF}(S) = \frac{|\mathcal{N}(S) \cap \mathcal{N}(D)|}{|\mathcal{N}(S)|}. \tag{9}$$

This is conservative: a number computed from two paper values (such as a relative improvement) is flagged even if correct, so flagged items go to a short manual review instead of being counted as errors automatically. NF = 1 if $\mathcal{N}(S) = \emptyset$, and years inside citations and list numbering are excluded together with slide numbers.

C Toy MMR Run

		MMR score $\lambda_m \mathrm{Rel} - (1-\lambda_m) \max \cos$			
Candidate	Rel(s_i)	Step 1	Step 2	Step 3	Outcome
s_1	0.82	**0.574**	–	–	picked first
s_2 (paraphrase of s_1)	0.78	0.546	0.276	0.276	rejected
s_3	0.64	0.448	**0.388**	–	picked second
s_4	0.55	0.385	0.355	**0.310**	picked third

Table 4: Toy MMR run with $\lambda_m = 0.7$. Similarities used: $\cos(s_2, s_1) = 0.90$, $\cos(s_3, s_1) = 0.20$, $\cos(s_4, s_1) = 0.10$, $\cos(s_2, s_3) = 0.15$, $\cos(s_4, s_3) = 0.25$. The near-duplicate s_2 has the second-highest relevance but is not chosen within the three picks. The numbers are illustrative, not measured.

D PrefSlides-Mini Construction Plan

Component	Plan	Size
Target papers	Open-access NLP/ML papers (e.g. ACL Anthology, CC BY, or arXiv papers with CC BY licence)	30
Reference pairs	Papers with publicly available author slides, licence checked, chosen for varied habits (result-heavy, concise, figure-heavy, etc.)	6
Templates	Free academic `.pptx` templates with different layouts and colours	4
Evaluation triples	Each target paper paired with 2 random (pair, template) choices; 10 triples form a development split	60
Manual section labels	All slides of the 6 reference decks labelled by two annotators; used to measure the tagger	≈70 slides
Support judgments	Manual check of whether the pointed passage supports the bullet: 5 random bullets from each of 10 test decks for each of 4 systems, judged by both annotators	200 bullets
Human ratings	Two annotators rate the same 10 test triples for the same 4 systems	40 decks

Table 5: PrefSlides-Mini construction plan. Where redistribution is not permitted, we release only links and our annotations.

E Evaluation Metrics

Metric	Definition	Assesses
PF $= 1 - D_{\text{pref}}$ and its parts	Eq. (5), computed by code	preference adherence, repair gain
Coverage (IoU), following [1]	$\frac{\lvert\mathcal{T}(S) \cap \mathcal{T}(S_{\text{ref}})\rvert}{\lvert\mathcal{T}(S) \cup \mathcal{T}(S_{\text{ref}})\rvert}$, with $\mathcal{T}(\cdot)$ the coarse topics a deck covers (taken from section labels in extractive mode, where it is therefore close to D_{alloc}; LLM extraction otherwise)	preference adherence
Flow, following [1]	$1 -$ NGLD between topic sequences [13]	preference adherence
Redundancy	mean pairwise TF-IDF cosine among the bullets of a deck (lower is better)	quality of extractive mode
Source support SR	Eq. (8), plus manual support rate on the sampled bullets and the agreement between the two	grounding, metric validity
Numeric fidelity NF	Eq. (9), with flagged items manually reviewed	grounding
Overflow O	Eq. (7)	text fit
Judge scores	Content structure and aesthetics, 1–5, cross-family LLM judge on slide text and rendered images	overall quality
Human ratings	Content accuracy, structure match, readability, 1–5, two raters, weighted κ [15]	accuracy, structure, readability
Cost and time	LLM calls and tokens per deck; processing time per paper; extra calls due to repair	cost

Table 6: Evaluation metrics.

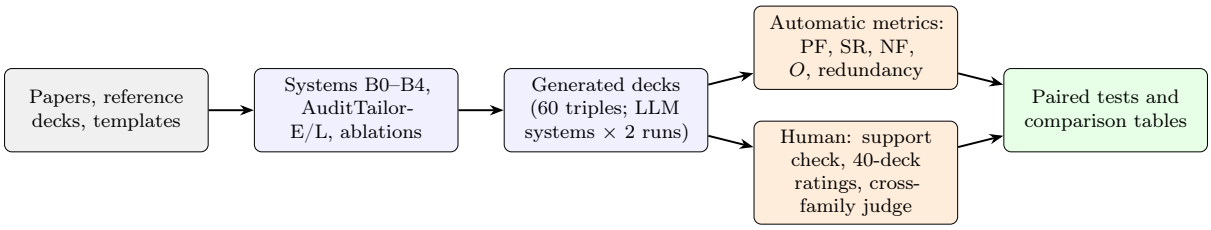


Figure 2: Evaluation workflow.

F Implementation Plan and Equation Map

The six phases are: (1) parsing with PyMuPDF; (2) profile and planner in Python and `python-pptx`; (3) extractive mode with scikit-learn, NumPy and `python-pptx`; (4) verifier in Python and Pillow; (5) LLM mode with a free-tier or local open model; (6) evaluation with LibreOffice rendering and SciPy. Table 7 maps each equation to its simplest feasible implementation.

Equation	Component	Simplest feasible implementation
(2)	Quantified profile	Count tagged slides with `python-pptx`; paragraphs per body placeholder; picture and table shape types; a regular expression for equation symbols; font metrics for η
(3)	Budgeted plan	A few lines of Python: the blend and the largest-remainder rounding
(4)	Extractive scoring and MMR	scikit-learn TF-IDF vectors and cosine similarity; a loop that picks one sentence at a time
Pointers	Source pointers	Reuse the same TF-IDF matrix: for each bullet, the paper sentence with the highest cosine; store page and section in the slide notes
(7)	Fit check	Pillow font metrics with greedy word wrapping to estimate text height; confirm on a sample by rendering to PDF
(5)	Preference distance	L_1 distance of share vectors, a small edit-distance routine for π, simple averages for style
(8), (9)	Support and numeric fidelity	A cosine threshold; a regular expression for numbers and a set intersection
(6)	Repair loop	The short loop of Algorithm 1 with rule-based fixes (drop, swap, re-sort, shorten) and, in LLM mode, optional LLM rewrites

Table 7: Where each equation is implemented, with the simplest feasible version of each component.